

Sistema de Reconocimiento de Marcha

Explicación Explícita de Cada Fase del Proyecto

Proyecto: ProyectoChino · 7 Fases · Fecha: 2026-05-06

Fases del Proyecto

#	Nombre	Script(s) Principal(es)	Estado
0	Auditoría del Dataset	audit_raw_dataset.py	Completada
1	Extracción de Siluetas	extract_all.py → extract_sequence.py	Completada
2	Empaquetado OpenGait	pack_to_pkl.py → merge_sessions → splits	Completada
3	Fine-tuning GaitBase	finetune_gaitbase_multisession.py	✓ Producción
4	Intento Multimodal	finetune_skeletongaitpp.py / train_stgcn.py	Descartada
5	Evaluación Open-set	openset_eval.py	Completada
6	Pipeline en Vivo	build_gallery.py / infer_live.py	Completada

FASE 0 — AUDITORÍA DEL DATASET

Objetivo: Antes de procesar nada, verificar el estado real del dataset crudo. Saber qué hay, qué falta y si la calidad es suficiente.

Script: `audit_raw_dataset.py`

- Recorre todos los videos .mp4 en el Dataset Crudo.
- Cuenta cuántas secuencias hay por sujeto y por condición (normal / rápido).
- Reporta cantidad de frames, fps y duración por video.
- Detecta videos corruptos, vacíos o con resolución inesperada.
- No modifica ningún archivo — solo lectura y reporte.

Entregable: reporte estadístico del dataset crudo. Este paso determinó que había 19 sujetos con 2 sesiones cada uno y aproximadamente 120 secuencias en total, todas a 1080p ~30 fps.

FASE 1 — EXTRACCIÓN DE SILUETAS Y KEYPOINTS

Objetivo: Convertir cada video .mp4 crudo en arrays numpy de siluetas binarias (64×44 px) y keypoints del esqueleto normalizados. Esta es la entrada que GaitBase necesita.

Scripts: `extract_all.py` (batch) → `extract_sequence.py` (por video)

Modelos utilizados:

Modelo	Entrada	Salida	Parámetros
YOLOv11n	Frame BGR completo	Bbox (x1,y1,x2,y2)	conf=0.35, iou=0.5 clases=[0] (persona)
RVM MobileNetV3	Frame RGB float [0,1]	Alpha matte (H,W) float [0,1]	downsample_ratio=0.25 estado recurrente entre frames
RTMPose-m (ONNX)	Frame BGR + bbox	17 keypoints COCO + confianzas	input_size=(192,256) backend=onnxruntime

Pipeline por frame (del código `extract_sequence.py`):

```
para cada frame del video:
1. bbox = YOLOv11n.predict(frame, classes=[0], conf=0.35)
→ selecciona la bbox de mayor área × confianza

2. alpha = RVM(frame_rgb_tensor, *rec, downsample_ratio=0.25)
→ alpha matte (H,W) float [0,1]
→ rec = estado recurrente actualizado (coherencia temporal)

3. sil = crop_and_normalize_silhouette(alpha, bbox):
a) recorta alpha al bbox + padding=5%
b) binariza: píxel > 0.5 → 255, resto → 0
c) calcula centroide horizontal de píxeles blancos
d) ajusta lienzo a aspect ratio 44/64=0.6875
- muy angosta → ensancha con ceros centrado en centroide
- muy ancha → recorta simétricamente al centroide
e) cv2.resize(INTER_AREA) → (64, 44) uint8

4. kp, scores = RTMPose(frame, bboxes=[bbox])
→ kp_raw: (17,3) en píxeles originales

5. kp_norm = normalize_keypoints(kp_raw):
→ centrar en cadera (promedio kp[11], kp[12])
→ escalar por longitud torso (cadera → hombro)
→ kp_norm: (17,3) float, x,y en unidades de torso
```

Detección del tramo útil (motion filtering):

```
# Solo guardar frames donde la persona está realmente caminando
MOTION_PX_THRESH = 6 # píxeles de desplazamiento mínimo entre frames
MOTION_WIN = 5 # frames consecutivos requeridos

motion[i] = ||centro_bbox[i] - centro_bbox[i-1]||■

start = primer i donde motion[i:i+5] > 6 px (todos)
end = último i donde motion[i-5:i] > 6 px (todos)

→ descarta frames iniciales (persona quieta entrando al cuadro)
→ descarta frames finales (persona quieta saliendo del cuadro)
```

Salidas por video (en `data/processed/sujeto/condición/`):

Archivo	Contenido	Shape
silhouettes.npy	Siluetas binarias del tramo útil	(T, 64, 44) uint8 ∈ {0,255}
keypoints.npy	Keypoints normalizados	(T, 17, 3) float32
keypoints_raw.npy	Keypoints en píxeles originales	(T, 17, 3) float32
bboxes.npy	Bounding boxes por frame	(T, 4) float32
meta.json	Parámetros usados + estadísticas	JSON
debug_silhouettes_grid.png	Grid 4×4 de 16 siluetas	PNG (para inspección visual)

FASE 2 — EMPAQUETADO A FORMATO OPENGAIT

Objetivo: Convertir los .npy de siluetas al formato de carpetas que el DataLoader de OpenGait espera, y fusionar las dos sesiones para el entrenamiento cross-session.

Paso 2a — pack_to_pkl.py:

- Lee silhouettes.npy de cada sujeto/condición.
- Verifica shape (T,64,44) — rechaza si no cumple.
- Guarda como .pkl con pickle.dump usando HIGHEST_PROTOCOL.
- Estructura de carpetas requerida por OpenGait:

```
data/pkl/  
/  
normal/  
090/ ← ángulo de cámara (090 = 90° lateral)  
seq00.pkl ← np.ndarray (T, 64, 44) uint8  
rapido/  
090/  
seq00.pkl
```

Nota: '090' es obligatorio en el formato OpenGait aunque solo haya un ángulo de cámara.

Paso 2b — merge_sessions_pkl.py:

- Toma data/pkl/ (sesión 1) y data/pkl_s2/ (sesión 2).
- Renombra los tipos añadiendo el tag de sesión: normal → normal_s1, normal_s2.
- Copia los .pkl a data/pkl_multisession/ manteniendo el label (sujeto) igual.
- Resultado: 4 secuencias por sujeto — normal_s1, normal_s2, rapido_s1, rapido_s2.

```
data/pkl_multisession/  
alexbojorquez/  
normal_s1/090/seq00.pkl ← s1, paso normal  
normal_s2/090/seq00.pkl ← s2, paso normal  
rapido_s1/090/seq00.pkl ← s1, paso rápido  
rapido_s2/090/seq00.pkl ← s2, paso rápido
```

El label 'alexbojorquez' es idéntico en s1 y s2.
TripletSampler puede mezclar sesiones en el mismo batch.

Paso 2c — define_splits.py + build_partition_json.py:

- Define el split subject-disjoint una sola vez con seed=42.
- 13 sujetos para train, 3 para validación, 3 para test.
- Guarda configs/splits.yaml y configs/partition_finetune.json.
- INMUTABLE: no se vuelve a modificar durante todo el proyecto.

Split	N	Sujetos
Train	13	jorgeespinoza, gilbertofelix, jesusantonioaguilarfelix, josefelix, carloscarrillo, juliouriarte, jesuscazare, robertocastaño, luislopez, brayannajera, cesarvasquez, carlospadilla, rodrigorui
Val	3	hectorsanchez, robertpereira, carlostorres
Test	3	jesusvalenzuela, alexbojorquez, ricardomora

FASE 3 — FINE-TUNING DE GAITBASE ✓ PRODUCCIÓN

Objetivo: Adaptar el modelo GaitBase preentrenado en 4,000 sujetos (Gait3D) para que distinga a los 19 sujetos del proyecto. El entrenamiento usa s1+s2 combinadas para que el modelo aprenda la dinámica de marcha y no la ropa.

Script: finetune_gaitbase_multisession.py

Construcción del modelo (del código):

```
class GaitBaseFT(nn.Module):
    Backbone = SetBlockWrapper(
        ResNet9(channels=[64,128,256,512], layers=[1,1,1,1],
        strides=[1,2,2,1], in_channel=1, maxpool=False)
    ) # aplica ResNet9 frame a frame
    FCs = SeparateFCs(in=512, out=256, parts_num=16)
    BNNecks = SeparateBNNecks(class_num=13, in=256, parts_num=16)
    TP = PackSequenceWrapper(torch.max) # pooling temporal
    HPP = HorizontalPoolingPyramid(bin_num=[16])

    forward(sils, seqL):
        outs = Backbone(sils) # (B, C, T, H, W)
        outs = TP(outs, seqL)[0] # max temporal → (B, 512, H', W')
        feat = HPP(outs) # 16 bandas horizontales
        embed_1 = FCs(feat) # (B, 256, 16)
        embed_2, logits = BNNecks(embed_1)
        return embed_1, logits
```

Carga de pesos preentrenados:

- Se carga GaitBase_Gait3D_120000.pt (entrenado en 4,000 sujetos).
- Se copian todos los pesos EXCEPTO BNNecks (la capa de clasificación).
- BNNecks se reinicializa aleatoriamente para 13 clases (en vez de 4,000).
- Esto preserva las representaciones de marcha aprendidas en Gait3D.

Hiperparámetros (de la clase Config en el código):

Parámetro	Valor	Justificación
seed	42	Reproducibilidad
batch_p (P)	4	4 identidades por batch
batch_k (K)	2	2 secuencias por identidad = 8 muestras/iter
frames_fixed	30	Ventana aleatoria de 30 frames por clip (data aug)
lr	0.01	Mismo que configuración original GaitBase
momentum	0.9	SGD estándar
weight_decay	5e-4	Regularización L2 para dataset pequeño
total_iter	1,500	Máximo; early stop activo
milestones	[750, 1250]	Decay lr × 0.1 en cada milestone
gamma	0.1	Factor de decay del lr
triplet_margin	0.2	Margen del TripletLoss
ce_scale	16.0	Escala logits antes del CrossEntropy
ce_smoothing	0.1	Label smoothing — evita sobreconfianza
early_stop_patience	3	Paradas sin mejora de margen NR val antes de detener

Parámetro	Valor	Justificación
eval_iter	200	Evaluar en val cada 200 iteraciones
AMP float16	Activado	GradScaler — necesario para GTX 1650 4 GB

Función de pérdida combinada:

```
Loss = TripletLoss(margin=0.2) + CrossEntropyLoss(scale=16, smooth=0.1)
```

TripletLoss:

- Calcula distancias euclidianas entre todos los pares del batch
- ap = distancia anchor-positive (misma identidad)
- an = distancia anchor-negative (distinta identidad)
- loss = max(0, ap - an + margin) por parte (16 bandas)
- Solo cuenta triplets activos (loss > 0)

CrossEntropyLoss:

- logits * scale=16 → logits amplificados
- label_smoothing=0.1 → evita probabilidad 1.0 en la clase correcta
- Se aplica sobre las 16 bandas × 13 clases

Early stopping (del código):

- Métrica de parada: margen medio NR (normal→rápido cross-session) en val.
- Margen = similitud_correcta – mejor_similitud_incorrecta.
- Si 3 evaluaciones consecutivas (cada 200 iter) no mejoran el margen → parar.
- Se guarda el estado del modelo en el mejor margen registrado.
- Resultado: mejor checkpoint en iteración 1,200 de 1,500 máximas.

Resultados del entrenamiento:

Métrica	Valor
Rank-1 NN (normal → normal)	94.4% (17/18 sujetos)
Rank-1 NR (normal → rápido)	94.7% (18/19 sujetos)
Rank-5 NN y NR	100%
Error único consistente	ricardomora ↔ hectorsanchez (mismo build corporal)
VRAM pico entrenamiento	1.25 GB
Tiempo total	~177 min en GTX 1650
Checkpoint de producción	gaitbase_ft_multisession_best_iter1200.pt

FASE 4 — INTENTO MULTIMODAL X DESCARTADA

Objetivo original: Mejorar las métricas fusionando silueta + pose (keypoints) en un modelo multimodal, como especificaba el requisito inicial.

Scripts probados:

- `finetune_skeletongaitpp.py` — SkeletonGait++ (AAAI 2024): rama de silueta + rama de skeleton map (heatmap de keypoints).
- `train_stgcn.py` — ST-GCN Lite: Graph Convolutional Network sobre keypoints temporales.
- `openset_eval_fusion.py` — evaluación de fusión tardía con peso α variable.

Protocolo de evaluación:

- Se optimizó el peso $\alpha \in [0.0, 1.0]$ donde $\text{score} = \alpha \cdot \text{sim_silueta} + (1-\alpha) \cdot \text{sim_pose}$.
- $\alpha=0.0$ significa solo pose, $\alpha=1.0$ significa solo silueta.
- Se buscó el α que maximizara $\text{TAR@FAR=0\%}$.

Experimento	α óptimo encontrado	EER	TAR@FAR=0%
SkeletonGait++ (fusión tardía)	1.0	8.11%	86.49%
ST-GCN Lite (fusión tardía)	1.0	8.11%	86.49%
Fusión pose manual GaitGraph2	0.4–0.7 (variable)	8.11%	86.49%

Conclusión: $\alpha^*=1.0$ en todos los experimentos. El optimizador asignó peso cero a la rama de pose. Las métricas son idénticas a GaitBase solo.

Cuatro razones técnicas del fracaso:

Razón	Explicación técnica
Ángulo lateral 90°	A 90°, las extremidades contralaterales se solapan en la imagen. Los keypoints de RTMPose no aportan dimensionalidad.
Dataset insuficiente para la rama de pose	Solo ~51 secuencias de entrenamiento (13 sujetos $\times$ ~4 tipos). Las redes de pose necesitan miles de ejemplos para aprender.
Solo 2 sesiones	El modelo de pose aprende la diferencia s1 vs s2 (velocidad, ropa) en vez de la dinámica real de marcha. SkeletonGait++ no aprende a reconocer la marcha.
Evidencia empírica directa	El optimizador nos dijo explícitamente que la pose no aporta. No es una hipótesis — es el resultado cuantitativo de la optimización.

Estado final de la pose en el pipeline:

- RTMPose sigue corriendo en el pipeline en vivo.
- Su función es quality gating: descartar frames donde la silueta extraída es de baja calidad (persona fuera de cuadro, oclusión severa).
- NO entra en el score de similitud de identidad.
- El requisito multimodal original se reinterpretó: 'pose se calcula pero no se fusiona en score' — decisión basada en evidencia.

FASE 5 — EVALUACIÓN OPEN-SET Y CALIBRACIÓN DE UMBRAL

Objetivo: El modelo en closed-set siempre devuelve una identidad. En producción hay que poder rechazar a personas que NO están en el sistema. Esta fase calibra el umbral τ que controla esa decisión.

Script: `openset_eval.py`

Protocolo LOSO cross-session (del código):

- Gallery: s1/normal de los 19 sujetos (18 válidos — falta `jesusantonioaguilarfelix/normal_s1` por problema de extracción).
- Probes: s2/normal (protocolo NN) y s2/rapido (protocolo NR).
- Para cada probe con identidad i:

GENUINO: query de persona *i* contra galería completa
→ se espera que `top1 = i` Y `sim >=  $\tau$` (aceptar)

IMPOSTOR: query de persona *i* contra galería SIN persona *i*
→ no importa quién sea `top1`, se espera `sim <  $\tau$` (rechazar)

Nota: el impostor es la misma persona pero evaluada como si fuera un desconocido. Simula a alguien que no está en el sistema.

Cálculo de métricas — barrido de τ (2,001 valores):

```
para cada  $\tau$  en linspace(sim_min, sim_max, 2001):  
TAR = (correct_top1 AND sim >=  $\tau$ ) / n_genuine  
FAR = (sim_top1_impostor >=  $\tau$ ) / n_impostor  
FRR = 1 - TAR
```

```
EER = punto donde |FAR - FRR| es mínimo  
= (FAR[eer_idx] + FRR[eer_idx]) / 2
```

Operating points calculados:

τ @ FAR=0% → mayor τ que garantiza cero falsos positivos

τ @ FAR≤1% → mayor τ con al menos 99% de impostores rechazados

τ @ FAR≤5% → mayor τ con al menos 95% de impostores rechazados

Resultados (protocolo ALL = NN + NR combinados):

Métrica	Valor
EER (Equal Error Rate)	8.11%
TAR @ FAR=0%	86.49% (32 de 37 genuinos identificados)
TAR @ FAR≤1%	86.49%
TAR @ FAR≤5%	86.49%
Umbral τ @ FAR=0%	0.9807 (→ ajustado a 0.9847 en pipeline)
Similitud media genuina	0.9883
Similitud media impostor	0.9666
Gap entre medias	+0.0217
N pares genuinos evaluados	37
N pares impostores evaluados	38

¿Por qué FAR=0% como restricción de diseño?

- En un sistema de asistencia, un falso positivo significa registrar como presente a alguien que no vino — inaceptable.
- Un falso negativo (no registrar a alguien que sí vino) es corregible manualmente. Un falso positivo no lo es.
- Por eso se sacrifica TAR (aceptamos que el 13.5% de los genuinos no sean registrados) a cambio de FAR=0%.

Salidas del script:

- reports/06_openset_.json — métricas completas y curva DET.
- reports/06_openset__pairs.csv — detalle de cada par (probe, top1, sim, correcto).
- reports/06_openset__roc.png — curva ROC con NN, NR y ALL.

FASE 6 — PIPELINE EN VIVO

Objetivo: Integrar todos los componentes en un sistema que corra en tiempo real con la cámara C920, identifique a las personas mientras caminan y registre la asistencia automáticamente.

Fase 6a — Construcción de la Galería (build_gallery.py):

- Carga el checkpoint de producción: gaitbase_ft_multisession_best_iter1200.pt
- Por cada uno de los 19 sujetos, toma hasta $K=4$ secuencias de referencia (s_1+s_2).
- Pasa cada secuencia por GaitBase $\rightarrow$ embedding de 256×16 dimensiones.
- Guarda los K embeddings por sujeto como templates de referencia.

```
gallery/embeddings.npy → (19, K_max, 256, 16) float32
gallery/valid_mask.npy → (19, K_max) bool
gallery/index.json → metadata (subject_id, session, n_templates, ...)

Estrategia de matching: max-sim por template (no promedio)
score(query, sujeto_i) = max( cosine(query, template_k) ) para k en templates_i
→ captura variabilidad intra-sujeto (s1 ≠ s2)
```

Fase 6b — Inferencia en Tiempo Real (infer_live.py):

El script abre la cámara C920 a 15 fps y encadena 8 módulos:

Módulo (src/pipeline/)	Función	Parámetros clave
capture.py	Lee frames de C920 con OpenCV	source=0, target_fps=15
detect.py	YOLOv11n: detecta personas, devuelve bbox mayor área	conf=0.35, iou=0.5
track.py	FSM 4 estados: IDLE→WARMING→ACTIVE→RESET	warmup=15 frames, reset=15 frames
segment.py	RVM: extrae silueta 64×44 (solo en ACTIVE)	downsample=0.25, resetea con FSM
seq_buffer.py	FIFO: acumula siluetas y emite secuencias	window=60 frames, stride=30
embed.py	GaitBase: secuencia → embedding 256×16	checkpoint iter1200, AMP float16
match.py	Cosine max-sim contra galería, aplica τ	τ =0.9847, gallery 19 sujetos
confirm.py	Exige N=2 matches consecutivos iguales	n_consecutive=2
log.py	Escribe CSV de asistencia	logs/attendance_YYYY-MM-DD.csv

Lógica del FSM Tracker (track.py):

[illegible]

Flujo completo de una identificación:

```
t=0s Persona entra al campo de visión
YOLO detecta bbox → FSM pasa a WARMING

t=1s 15 frames consecutivos con bbox → FSM pasa a ACTIVE
RVM empieza a extraer siluetas → buffer se llena

t=3s Buffer llega a 60 frames → primera secuencia emitida
GaitBase genera embedding → matcher devuelve top1 + similitud
Si sim >= τ=0.9847 → confirmación 1/2

t=5s Buffer acumula 30 frames más (stride) → segunda secuencia
Si mismo top1 con sim >= τ → confirmación 2/2
→ IDENTIDAD CONFIRMADA
→ Banner en UI: '✓ alexbojorquez (sim=0.9851)'
→ Logger escribe fila en CSV

Tiempo total desde entrada hasta confirmación: ~4-5 segundos
```

Análisis de latencia por componente:

Componente	Latencia típica	Nota
Captura de frame	~67 ms	1/15 fps
YOLOv11n	10–20 ms	GPU, batch=1
RVM MobileNetV3	30–40 ms	GPU, downsample=0.25
GaitBase embedding	80–100 ms	GPU, AMP float16, se ejecuta cada 30 frames
Matching (19 sujetos)	<1 ms	NumPy, CPU
Total por frame	~130–160 ms	≈7-8 fps de procesamiento efectivo
VRAM total pipeline	~3.5 GB pico	YOLO + RVM + GaitBase simultáneos

Salida: CSV de asistencia diario:

```
logs/attendance_2026-05-03.csv

timestamp,subject,mean_sim,n_sequences,frames_used,session_id
2026-05-03 10:15:23,alexbojorquez,0.9851,2,120,sess_abc123
2026-05-03 10:18:45,carlostorres,0.9834,2,120,sess_def456
```

Hotkeys del sistema en vivo:

- q → salir del sistema.
- r → reset manual (fuerza vuelta a IDLE sin esperar 15 frames).
- f → toggle pantalla completa.

RESUMEN EJECUTIVO DEL PROYECTO

Fase	Entregable Principal	Resultado
0 — Auditoría	Reporte de calidad del dataset	19 sujetos, ~120 seqs, OK
1 — Extracción	silhouettes.npy (T,64,44) por video	~120 secuencias procesadas
2 — Empaquetado	pkl_multisession/ + splits.yaml	4 seqs/sujeto, 13/3/3 split
3 — Fine-tuning	gaitbase_ft_multisession_best_iter1200.pt	Rank-1: 94.5% EER: 8.11%
4 — Multimodal	Experimentos descartados ($\alpha^*=1.0$)	Silueta sola = óptimo
5 — Open-set	$\tau=0.9847$ TAR@FAR=0%=86.49%	FAR=0% garantizado
6 — Pipeline Vivo	infer_live.py + CSV de asistencia	~4s por identificación

Limitaciones documentadas y aceptadas:

- ricardomora $\leftrightarrow$ hectorsanchez: mismo build corporal, indistinguibles a 90° lateral.
- Dataset pequeño (19 sujetos, 2 sesiones): la pose no puede aprovecharse.
- Un solo ángulo (90°): cambio de posición de cámara requiere recalibración.
- Cambio de ropa extremo puede degradar similitud por debajo de τ .

Trabajo futuro (Fase 7):

- Quantización INT8 / TensorRT para reducir latencia de embedding a ~30 ms.
- Reemplazar CSV por SQLite para log persistente y consultable.
- Soporte multi-cámara y multi-ángulo.
- Recalibración automática de τ con nuevas sesiones de grabación.